Predicción de volatilidad financiera

Apuntes teóricos y computacionales para la Semana 5

Rendimientos logarítmicos, absolutos y al cuadrado

Objetivo de la semana. Transformar la serie limpia de precios de `SPY` construida en la Semana 4 en una base de rendimientos financieros. Se calcularán rendimientos logarítmicos, rendimientos absolutos y rendimientos al cuadrado, se estudiará su interpretación financiera y estadística, se realizarán validaciones automáticas y se exportará una nueva base procesada que servirá como entrada para el cálculo de volatilidad histórica en la Semana 6.

Índice

1. Contexto de la Semana 5 **4**

2. Objetivos de aprendizaje **4**

3. Del precio al rendimiento **5**

3.1. Rendimiento simple 5

4. Rendimiento logarítmico **5**

4.1. Interpretación del signo 6

4.2. Relación con el rendimiento simple 6

4.3. Propiedad aditiva 6

5. Por qué usar `Adj Close` **7**

6. Lectura de la base limpia **7**

7. Cálculo del rendimiento logarítmico **8**

8. El primer valor faltante no es un error **9**

9. Rendimiento absoluto **9**

10. Rendimiento al cuadrado **10**

11. Comparación de las tres variables **10**

12. Ejemplo manual **11**

13. Comparación entre rendimiento simple y logarítmico **11**

14. Estadística descriptiva **12**

15. Visualización de los rendimientos **13**

16. Visualización de magnitudes **13**

17. Histograma de rendimientos **14**

18. Identificación de los mayores movimientos **14**

19. Validaciones automáticas **15**

19.1. No negatividad 15

19.2. Consistencia con las definiciones 15

19.3. Valores finitos 15

20. Construcción de la base final de la semana **15**

21. Ejercicios **17**

22. Checklist de cierre **17**

23. Producto esperado **18**

1. Contexto de la Semana 5

En la Semana 4 se construyó una base limpia, ordenada y validada de `SPY`, almacenada en

`data/processed/SPY_2015_2025_clean.csv`.

Esa base conserva las variables de mercado

`Open`, `High`, `Low`, `Close`, `Adj Close`, `Volume`,

y satisface las condiciones básicas necesarias para continuar con el análisis cuantitativo.

El precio por sí solo no es la variable más adecuada para estudiar variabilidad financiera. Dos activos con precios muy diferentes pueden experimentar cambios absolutos de magnitud distinta y, sin embargo, movimientos relativos comparables. Por ello, a partir de esta semana el análisis se centrará principalmente en los *rendimientos*.

La secuencia de trabajo será

base limpia de precios $\longrightarrow$ rendimientos $\longrightarrow$ transformaciones de magnitud $\longrightarrow$ base de rendimientos.

> **Delimitación de la semana.** En esta etapa todavía no se calculará volatilidad histórica mediante ventanas móviles. Tampoco se construirán rezagos, variables predictoras, EWMA o modelos GARCH. Estos elementos corresponden a semanas posteriores.

2. Objetivos de aprendizaje

Al finalizar la semana, la estudiante deberá ser capaz de:

1. distinguir entre precio, cambio absoluto y rendimiento;
2. calcular e interpretar rendimientos simples;
3. calcular rendimientos logarítmicos a partir de precios ajustados;
4. explicar por qué el rendimiento logarítmico es conveniente en series financieras;
5. interpretar el signo y la magnitud de un rendimiento;
6. construir rendimientos absolutos y rendimientos al cuadrado;
7. explicar por qué $|r_t|$ y r_t^2 son aproximaciones simples de la magnitud de los movimientos del mercado;
8. identificar correctamente el valor faltante inducido por la operación de diferencia;
9. obtener estadísticas descriptivas de los rendimientos;

10. interpretar asimetría y curtosis de manera básica;
11. visualizar rendimientos y movimientos extremos;
12. identificar las fechas con mayor magnitud de cambio;
13. validar computacionalmente las transformaciones realizadas;
14. exportar una nueva base reproducible para la Semana 6.

3. Del precio al rendimiento

Sea $P_t > 0$ el precio de un activo financiero en el día t. El cambio absoluto entre dos días consecutivos es

$$\Delta P_t = P_t - P_{t-1}.$$

Esta cantidad conserva información útil, pero depende directamente de la escala del precio. Por ejemplo, un aumento de 5 dólares puede ser muy grande para un activo cuyo precio es 20 dólares y pequeño para otro cuyo precio es 500 dólares.

Por esta razón suele ser más útil estudiar cambios relativos.

3.1. Rendimiento simple

El rendimiento simple entre los tiempos $t-1$ y t se define por

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}} = \frac{P_t}{P_{t-1}} - 1.$$

Si $R_t = 0.02$, el precio aumentó aproximadamente un $2\,\%$ respecto al día anterior. Si $R_t = -0.02$, disminuyó aproximadamente un $2\,\%$.

En Python puede calcularse mediante

```
rend_simple = datos["Adj Close"].pct_change()
```

El rendimiento simple se utilizará en esta semana principalmente con fines de comparación. La variable principal del proyecto será el rendimiento logarítmico.

4. Rendimiento logarítmico

El rendimiento logarítmico se define como

$$r_t = \log\left(\frac{P_t}{P_{t-1}}\right).$$

Utilizando propiedades del logaritmo,

$$r_t = \log(P_t) - \log(P_{t-1}).$$

Esta segunda expresión muestra que el rendimiento logarítmico es simplemente la primera diferencia de la serie $\log(P_t)$.

4.1. Interpretación del signo

Como el logaritmo es una función creciente,

$$P_t > P_{t-1} \quad \Longrightarrow \quad r_t > 0,$$
$$P_t < P_{t-1} \quad \Longrightarrow \quad r_t < 0,$$

y

$$P_t = P_{t-1} \quad \Longrightarrow \quad r_t = 0.$$

Por tanto, el signo de r_t indica la dirección del movimiento, mientras que $|r_t|$ indica su magnitud aproximada.

4.2. Relación con el rendimiento simple

Dado que

$$\frac{P_t}{P_{t-1}} = 1 + R_t,$$

se tiene

$$r_t = \log(1 + R_t).$$

Para valores pequeños de R_t, la aproximación de Taylor

$$\log(1 + x) \approx x$$

implica

$$r_t \approx R_t.$$

Por ello, para movimientos diarios moderados ambos rendimientos suelen ser numéricamente cercanos.

4.3. Propiedad aditiva

Una ventaja importante del rendimiento logarítmico es su aditividad temporal. Si se acumulan k días,

$$\sum_{j=1}^{k} r_{t+j} = \log\left(\frac{P_{t+k}}{P_t}\right).$$

Esta propiedad facilita el trabajo con horizontes temporales diferentes.

5. Por qué usar `Adj Close`

En este proyecto se utilizará `Adj Close` como serie principal de precios para calcular rendimientos.

El precio de cierre ajustado busca reflejar modificaciones históricas asociadas con eventos corporativos, por ejemplo divisiones de acciones y distribuciones. Esto permite construir una serie más consistente para comparar precios a lo largo del tiempo.

La condición indispensable para aplicar el logaritmo es

$$P_t > 0.$$

Antes de calcular rendimientos se verificará explícitamente:

```
if (datos["Adj Close"] <= 0).any():
    raise ValueError(
        "Adj Close contiene valores no positivos."
    )
```

Importante. El logaritmo de un precio nulo o negativo no está definido en los números reales. Por ello, la validación de positividad realizada en la Semana 4 no es solo una revisión de calidad: es una condición matemática necesaria para esta transformación.

6. Lectura de la base limpia

El notebook de esta semana puede llamarse

`05_rendimientos_financieros.ipynb`.

Se utilizarán las bibliotecas

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path
```

Las rutas principales serán

```
ruta_processed = Path("data/processed")

archivo_clean = (
    ruta_processed / "SPY_2015_2025_clean.csv"
)

archivo_returns = (
    ruta_processed / "SPY_2015_2025_returns.csv"
)
```

Antes de leer la base se verifica que exista:

```
if not archivo_clean.exists():
    raise FileNotFoundError(
        f"No se encontro el archivo: {archivo_clean}"
    )
```

La lectura se realiza con

```
datos_clean = pd.read_csv(
    archivo_clean,
    index_col=0,
    parse_dates=True
)

datos = datos_clean.copy(deep=True)
```

Después conviene verificar

```
print(datos.shape)
print(datos.index.min())
print(datos.index.max())
print(datos.columns.tolist())
datos.head()
```

7. Cálculo del rendimiento logarítmico

La forma más directa de implementar

$$r_t = \log(P_t) - \log(P_{t-1})$$

es

```
datos["Return"] = np.log(
    datos["Adj Close"]
).diff()
```

También puede escribirse

```
rend_log_alt = np.log(
    datos["Adj Close"] /
    datos["Adj Close"].shift(1)
)
```

Ambas expresiones deben producir los mismos valores, salvo diferencias numéricas extremadamente pequeñas debidas a la representación de números reales en la computadora.

La equivalencia puede comprobarse mediante

```
comparables = pd.concat(
    [datos["Return"], rend_log_alt],
```

```
    axis=1
).dropna()

assert np.allclose(
    comparables.iloc[:, 0],
    comparables.iloc[:, 1]
)
```

8. El primer valor faltante no es un error

Al aplicar `diff()` o `shift(1)`, la primera observación del rendimiento es necesariamente faltante. Para calcular

$$r_1 = \log(P_1) - \log(P_0)$$

se necesitaría conocer el precio P_0, pero la base comienza en P_1.

Por ello, después del cálculo se espera

```
print(datos[["Adj Close", "Return"]].head())
print(datos["Return"].isna().sum())
```

Si la serie de precios está completa, el único valor faltante inducido por esta operación debe ser el primero.

Puede comprobarse mediante

```
assert datos["Return"].isna().sum() == 1
assert pd.isna(datos["Return"].iloc[0])
```

Después de construir todas las transformaciones de esta semana, se eliminará esa primera fila de la base de rendimientos.

> **Interpretación correcta.** Este `NaN` no representa una falla de Yahoo Finance ni una observación perdida. Es una consecuencia matemática de calcular una diferencia temporal.

9. Rendimiento absoluto

El rendimiento absoluto se define por

$$a_t = |r_t|.$$

En Python:

```
datos["Abs_Return"] = datos["Return"].abs()
```

La transformación elimina la dirección del movimiento y conserva únicamente su magnitud.

Por ejemplo, si

$$r_t = 0.03, \qquad r_s = -0.03,$$

entonces

$$|r_t| = |r_s| = 0.03.$$

Desde el punto de vista de la variabilidad, ambos días presentan movimientos de magnitud comparable, aunque uno sea positivo y el otro negativo.

El rendimiento absoluto se utiliza frecuentemente como una medida sencilla de intensidad de los movimientos financieros.

10. Rendimiento al cuadrado

El rendimiento al cuadrado se define como

$$q_t = r_t^2.$$

En Python:

```
datos["Sq_Return"] = datos["Return"] ** 2
```

Al igual que $|r_t|$, esta transformación elimina el signo. Sin embargo, el cuadrado da un peso relativo mayor a los movimientos grandes.

Por ejemplo,

$$(0.01)^2 = 0.0001, \qquad (0.04)^2 = 0.0016.$$

Aunque 0.04 es cuatro veces 0.01, su cuadrado es dieciséis veces mayor. Por ello, r_t^2 resalta con mayor intensidad los episodios extremos.

Esta variable será particularmente importante más adelante, ya que los modelos ARCH y GARCH utilizan explícitamente innovaciones al cuadrado para modelar la varianza condicional.

11. Comparación de las tres variables

Variable	Conserva signo	Es no negativa	Interpretación principal
r_t	Sí	No	Dirección y magnitud
$\|r_t\|$	No	Sí	Magnitud del movimiento
r_t^2	No	Sí	Magnitud con énfasis en extremos

Las tres variables contienen información relacionada, pero no equivalente. El rendimiento conserva la dirección; las otras dos transformaciones se concentran en la intensidad del movimiento.

12. Ejemplo manual

Supóngase que

$$P_{t-1} = 100, \qquad P_t = 103.$$

El rendimiento simple es

$$R_t = \frac{103}{100} - 1 = 0.03.$$

El rendimiento logarítmico es

$$r_t = \log(1.03) \approx 0.02956.$$

Por tanto,

$$|r_t| \approx 0.02956,$$

y

$$r_t^2 \approx 0.000874.$$

Ahora considérese una caída de 100 a 97:

$$r_t = \log(0.97) \approx -0.03046.$$

En este caso,

$$|r_t| \approx 0.03046, \qquad r_t^2 \approx 0.000928.$$

El signo permite distinguir subida y caída, mientras que las transformaciones de magnitud muestran que ambos movimientos son similares en tamaño.

13. Comparación entre rendimiento simple y logarítmico

Puede construirse una tabla de comparación:

```
rend_simple = datos["Adj Close"].pct_change()

comparacion = pd.DataFrame({
    "Simple": rend_simple,
    "Logaritmico": datos["Return"]
})

comparacion["Diferencia"] = (
    comparacion["Simple"] -
    comparacion["Logaritmico"]
)

comparacion.dropna().head(10)
```

También puede calcularse la mayor diferencia absoluta:

```
max_diferencia = (
    comparacion["Diferencia"]
    .abs()
    .max()
)

print(max_diferencia)
```

La diferencia no debe interpretarse como un error. Son definiciones distintas que se aproximan entre sí cuando los cambios diarios son pequeños.

14. Estadística descriptiva

Después de construir las tres variables, puede obtenerse un resumen mediante

```
variables = [
    "Return",
    "Abs_Return",
    "Sq_Return"
]

resumen = datos[variables].describe().T
resumen
```

La tabla contiene, entre otras cantidades:

- número de observaciones;
- media;
- desviación estándar;
- mínimo;
- cuartiles;
- máximo.

Para el rendimiento logarítmico también se pueden calcular dos medidas adicionales:

```
asimetria = datos["Return"].skew()
curtosis_exceso = datos["Return"].kurt()

print("Asimetria:", asimetria)
print("Exceso de curtosis:", curtosis_exceso)
```

La asimetría ofrece una descripción básica de la falta de simetría de la distribución. La función `kurt()` de `pandas` reporta exceso de curtosis: una distribución normal tiene valor de referencia cercano a cero bajo esta convención.

Alcance. Estas estadísticas son descriptivas. En esta semana no se realizará todavía una prueba formal de normalidad ni se supondrá que los rendimientos siguen una distribución específica.

15. Visualización de los rendimientos

Una gráfica temporal de r_t permite identificar cambios de magnitud variable a lo largo del periodo:

```
fig, ax = plt.subplots(figsize=(12, 4))
ax.plot(datos.index, datos["Return"], linewidth=0.8)
ax.axhline(0, linewidth=0.8)
ax.set_title("Rendimientos logaritmicos diarios de SPY")
ax.set_xlabel("Fecha")
ax.set_ylabel("Rendimiento")
plt.show()
```

El objetivo no es explicar aún cada episodio, sino reconocer visualmente dos características:

- los rendimientos fluctúan alrededor de valores relativamente pequeños;
- existen periodos en los que movimientos grandes aparecen agrupados en el tiempo.

Esta segunda observación anticipa de manera intuitiva el fenómeno de agrupamiento de volatilidad, que será estudiado con más detalle posteriormente.

16. Visualización de magnitudes

El rendimiento absoluto puede graficarse mediante

```
fig, ax = plt.subplots(figsize=(12, 4))
ax.plot(datos.index, datos["Abs_Return"], linewidth=0.8)
ax.set_title("Rendimientos absolutos diarios de SPY")
ax.set_xlabel("Fecha")
ax.set_ylabel("|r_t|")
plt.show()
```

El rendimiento al cuadrado:

```
fig, ax = plt.subplots(figsize=(12, 4))
ax.plot(datos.index, datos["Sq_Return"], linewidth=0.8)
ax.set_title("Rendimientos al cuadrado diarios de SPY")
ax.set_xlabel("Fecha")
ax.set_ylabel("r_t^2")
plt.show()
```

Al eliminar el signo, ambas gráficas permiten observar con mayor claridad los periodos de movimientos intensos.

17. Histograma de rendimientos

La distribución empírica puede visualizarse mediante

```
fig, ax = plt.subplots(figsize=(8, 4))
ax.hist(
    datos["Return"].dropna(),
    bins=60,
    edgecolor="black",
    alpha=0.8
)
ax.set_title("Distribucion de rendimientos logaritmicos")
ax.set_xlabel("Rendimiento")
ax.set_ylabel("Frecuencia")
plt.show()
```

El histograma permite observar la concentración de muchos rendimientos cerca de cero y la existencia de movimientos positivos y negativos de mayor magnitud.

Un movimiento extremo no debe eliminarse simplemente por encontrarse lejos del centro de la distribución. Puede representar información financiera real y será precisamente relevante para el estudio de volatilidad.

18. Identificación de los mayores movimientos

Una forma directa de localizar las observaciones con mayor magnitud es ordenar por `Abs_Return`:

```
mayores_movimientos = (
    datos[
        [
            "Adj Close",
            "Return",
            "Abs_Return",
            "Sq_Return"
        ]
    ]
    .dropna()
    .nlargest(10, "Abs_Return")
)

mayores_movimientos
```

Esta tabla responde a la pregunta:

> *¿Cuáles fueron los diez días con mayor movimiento porcentual aproximado en la muestra?*

En esta etapa basta con identificar y describir las fechas. La interpretación histórica de eventos específicos puede reservarse para una fase posterior del proyecto.

19. Validaciones automáticas

Las transformaciones de esta semana admiten comprobaciones sencillas y útiles.

19.1. No negatividad

Por definición,

$$|r_t| \geq 0, \qquad r_t^2 \geq 0.$$

Por tanto:

```
validos = datos["Return"].notna()

assert (
    datos.loc[validos, "Abs_Return"] >= 0
).all()

assert (
    datos.loc[validos, "Sq_Return"] >= 0
).all()
```

19.2. Consistencia con las definiciones

```
assert np.allclose(
    datos.loc[validos, "Abs_Return"],
    np.abs(datos.loc[validos, "Return"])
)

assert np.allclose(
    datos.loc[validos, "Sq_Return"],
    datos.loc[validos, "Return"] ** 2
)
```

19.3. Valores finitos

```
assert np.isfinite(
    datos.loc[validos, "Return"]
).all()
```

Estas condiciones convierten definiciones matemáticas en pruebas automáticas del pipeline.

20. Construcción de la base final de la semana

Una vez creadas las variables, se elimina únicamente la primera fila inducida por la diferencia temporal:

```
datos_returns = datos.dropna(
    subset=[
        "Return",
        "Abs_Return",
        "Sq_Return"
    ]
).copy()
```

La base final conserva

```
columnas_finales = [
    "Open",
    "High",
    "Low",
    "Close",
    "Adj Close",
    "Volume",
    "Return",
    "Abs_Return",
    "Sq_Return"
]

datos_returns = datos_returns[columnas_finales]
```

La exportación se realiza mediante

```
datos_returns.to_csv(
    archivo_returns,
    index=True
)
```

Se recomienda volver a leer el archivo y verificar su estructura:

```
verificacion = pd.read_csv(
    archivo_returns,
    index_col=0,
    parse_dates=True
)

assert verificacion.shape == datos_returns.shape
assert verificacion.index.is_monotonic_increasing
assert not verificacion["Return"].isna().any()
```

La estructura del pipeline queda entonces

$$\texttt{raw} \longrightarrow \texttt{clean} \longrightarrow \texttt{returns}.$$

21. Ejercicios

1. Calcule manualmente el rendimiento simple y el rendimiento logarítmico para tres pares consecutivos de precios de la base y verifique los resultados con Python.
2. Construya una tabla con `Adj Close`, `Return`, `Abs_Return` y `Sq_Return` para las primeras diez fechas válidas.
3. Calcule la diferencia absoluta entre el rendimiento simple y el logarítmico. Identifique la fecha en que esa diferencia es máxima.
4. Encuentre los diez rendimientos positivos más grandes y los diez rendimientos negativos más pequeños.
5. Calcule media, mediana y desviación estándar de `Return`. Compare la media con la desviación estándar y escriba una interpretación breve.
6. Calcule los percentiles 1, 5, 25, 50, 75, 95 y 99 del rendimiento logarítmico.
7. Grafique por separado `Return`, `Abs_Return` y `Sq_Return`. Identifique visualmente al menos dos periodos con movimientos relativamente intensos.
8. Construya un histograma de `Return` usando diferentes números de clases, por ejemplo 30, 60 y 100. Describa qué características permanecen estables.

22. Checklist de cierre

Antes de considerar terminada la Semana 5, debe verificarse que:

- la base de entrada es `SPY_2015_2025_clean.csv`;
- `Adj Close` es estrictamente positivo;
- `Return` se calculó mediante diferencias logarítmicas;
- el único faltante inducido inicialmente es la primera observación;
- `Abs_Return` coincide con $|r_t|$;
- `Sq_Return` coincide con r_t^2;
- se obtuvieron estadísticas descriptivas;
- se generaron las gráficas principales;
- se identificaron los mayores movimientos;
- la base final no contiene faltantes en las tres variables nuevas;
- el archivo `SPY_2015_2025_returns.csv` fue exportado y verificado.

23. Producto esperado

Al finalizar la semana se deberán tener los siguientes productos:

1. notebook `05_rendimientos_financieros.ipynb`;
2. base `data/processed/SPY_2015_2025_returns.csv`;
3. tabla descriptiva de r_t, $|r_t|$ y r_t^2;
4. gráfica temporal de rendimientos;
5. gráfica de rendimientos absolutos;
6. gráfica de rendimientos al cuadrado;
7. histograma de rendimientos;
8. tabla con los diez mayores movimientos según $|r_t|$;
9. respuestas breves a las preguntas de interpretación.

Conexión con la Semana 6. La base construida aquí contiene las transformaciones elementales necesarias para estudiar variabilidad. En la siguiente semana se calculará volatilidad histórica mediante ventanas móviles de 5, 10, 20 y 30 días, manteniendo siempre el orden temporal de la información.